# DELHI METRO ROUTE APP

Product Requirements Document (PRD)

Version 1.0

**Document Information**

| | |
|---|---|
| | |
| | |
| | |

## Table of Contents



## 1. Executive Summary

Delhi Metro Route App is an Android mobile application designed to help commuters navigate the Delhi Metro Rail System efficiently. The app provides intelligent route planning, real-time updates, and comprehensive station information to make the metro experience seamless and user-friendly.

### Problem Statement

- Commuters struggle to find optimal routes between stations, especially with multiple line options
- Lack of clear guidance on interchange points and platform selection
- No easy way to check metro operating hours, facility information, or crowd status
- Missing real-time delay or closure updates affecting commute planning

### Solution Overview

A lightweight, intuitive Android app that leverages the Delhi Metro's latest network map to provide two competing route options: the fastest route and the route with the least interchanges. The app includes real-time updates, comprehensive station information (operating hours, facilities, crowd levels), offline support, and theme customization using Catppuccin color schemes.

### Target Audience

- Daily metro commuters (office workers, students)
- Occasional visitors to Delhi
- Tourists seeking metro navigation assistance
- Tech-savvy users aged 16–55

## 2. Product Vision & Scope

### Vision Statement

To become the most reliable and user-friendly route planning tool for Delhi Metro commuters, reducing commute planning friction and helping users make informed travel decisions through intelligent routing and real-time intelligence.

### Core Principles

- **Simplicity:** Minimal inputs, maximum clarity
- **Speed:** Instant route calculation and retrieval
- **Accuracy:** Current Delhi Metro network data
- **Accessibility:** Clear typography, color-coded lines, intuitive UX
- **Reliability:** Works offline and syncs when online

### In Scope

- Route planning with two options (fastest & least interchanges)
- Real-time metro updates (delays, closures)
- Station information (hours, facilities, exit maps, crowd levels)
- Fare calculation based on distance
- Offline mode with cached metro map
- Theme support (Catppuccin Mocha & Latte)

- Alternative transport suggestions (when metro unavailable)
- Color-coded line indicators for visual clarity

### Out of Scope (Phase 1)

- Favorite/saved routes
- User ratings/feedback system
- Social features (reviews, community tips)
- Integration with other transport modes (buses, autos)
- Accessibility features for people with disabilities
- Multi-language support

## 3. Core Features

### 3.1 Route Planning

#### Dual Route Options

**Input Fields:**

- Starting Station (searchable dropdown)
- Ending Station (searchable dropdown)

**Route Output:**

- Route 1: Fastest Route
- Route 2: Least Interchanges

**Route Display Format:**

- Horizontal or vertical scrollable list of stations
- Station names with line color indicators (left dot/badge)
- Highlight interchange stations distinctly
- Show platform numbers for each segment
- Estimated time per segment

### 3.2 Real-Time Metro Updates

- Display alerts for line closures, delays, or schedule changes
- If no route exists due to closures, suggest alternative transport modes (bus, auto-rickshaw, taxi)
- Show estimated service resumption time
- Sync updates when connected; cache locally for offline access

### 3.3 Station Information

- Tap any station to view details:
    - Operating hours (first train & last train timings)

    - Available facilities (ATM, restroom, parking, ticket counter)
    - Exit maps with landmark references
    - Crowd level indicator (sparse, moderate, crowded)

### 3.4 Fare Calculator

- Automatically calculate fare based on number of stations
- Display in both single-journey and card options
- Show savings with travel card vs. single journey

### 3.5 Offline Mode

- Cache entire metro network data locally on first launch
- Route calculation works without internet connection
- Real-time updates sync when network is available
- Display "offline mode" badge and last sync timestamp

### 3.6 Theme Customization

- Two built-in themes:
    - Catppuccin Mocha (dark theme)
    - Catppuccin Latte (light theme)
- Toggle in Settings menu
- Persist theme preference across sessions

## 4. User Interface & Design

### 4.1 Design Philosophy

The app prioritizes clarity, speed, and ease of use. Every element serves a functional purpose. The interface is optimized for quick interactions while maintaining aesthetic appeal through the Catppuccin color palette.

### 4.2 Typography

**Font Family:** Sans-serif (e.g., Roboto, Inter, or system default)

**Heading 1 (H1):** 28–32 sp, bold, primary color

**Heading 2 (H2):** 20–24 sp, bold, secondary color

**Body Text:** 14–16 sp, regular, readable contrast

**Labels & Small Text:** 12–13 sp, secondary text color

**Line Height:** 1.5x for body text, 1.2x for headings

### 4.3 Color Palette (Catppuccin Mocha & Latte)

Metro lines use official colors; UI elements use Catppuccin palette for consistency.

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## 4.4 Screen Layouts

### Home Screen

- Large searchable input fields for "From" and "To" stations
- Swap icon to reverse stations
- Search button to initiate route calculation
- Settings icon (top-right) for theme and options
- Recent searches (if applicable, for future phases)

### Route Results Screen

- Two tabbed views: "Fastest" and "Least Interchanges"
- Route summary card showing: total time, station count, fare
- Scrollable station list with:
    - Colored dot (line color) on the left
    - Station name
    - Time to next station (if first segment)
    - "Interchange" label for transfer points
- Tap any station for detailed info (hours, facilities, exit map, crowd level)
- Back button to return to search

### Station Detail Screen

- Station name and line color badge
- Operating hours (first/last train timings)
- Available facilities with icons (ATM, restroom, etc.)
- Exit map image/thumbnail
- Current crowd level indicator
- Back button

### Settings Screen

- Theme selector (toggle between Mocha & Latte)
- App version & build info
- Cache management (view size, clear cache option)

- Last sync timestamp
- About / Credits

# 5. Technical Requirements

## 5.1 Platform & Requirements

**Platform:** Android 8.0 (API level 26) or higher

**Min RAM:** 2 GB

**Storage:** 50–100 MB for app + cached metro data

**Network:** WiFi or mobile data (for real-time updates)

## 5.2 Technology Stack

**Language:** Kotlin (primary) or Java

**Framework:** Android SDK with Jetpack Compose or XML layouts

**Database:** SQLite for local caching, Room ORM

**Networking:** Retrofit + OkHttp for API calls

**Async:** Coroutines for background operations

**DI:** Hilt for dependency injection

**UI Framework:** Jetpack Compose (modern) or Material Design 3

**Theming:** Material 3 with Catppuccin color palettes

## 5.3 Performance Requirements

- Route calculation: < 500 ms
- App launch time: < 2 seconds
- Network requests: Timeout after 10 seconds
- Memory footprint: < 100 MB
- Smooth 60 FPS scrolling in all lists

## 5.4 Offline Capability

- All metro network data cached locally on first app launch
- Route calculation works without internet
- Real-time update data synced when network available
- Offline mode visually indicated to user
- Last sync timestamp displayed

# 6. Data Models & API

## 6.1 Core Data Models

### Station

{ id, name, lineId, lineColor, latitude, longitude, platformNumber, operatingHours (firstTrain, lastTrain), facilities [], exitMap, currentCrowdLevel }

### MetroLine

{ id, name (e.g., 'Blue Line', 'Red Line'), color (hex), stations [] }

### Route

{ id, sourceStationId, destStationId, routeType ('fastest' | 'least_interchanges'), stations [], interchanges [], totalTime (minutes), totalStations, fare (₹), segments [] }

### RouteSegment

{ startStationId, endStationId, lineId, durationMinutes, platformFrom, platformTo, direction }

### MetroUpdate

{ id, lineId, type ('delay' | 'closure' | 'scheduled_maintenance'), message, affectedStations [], severity, estimatedResumption, timestamp }

## 6.2 API Endpoints

### GET /api/v1/metro-network

**Description:** Fetch complete metro network (lines, stations)

**Response:** { lines: [], stations: [] }

### POST /api/v1/routes/calculate

**Description:** Calculate dual routes (fastest & least interchanges)

**Request:** { sourceStationId, destStationId }

**Response:** { fastest: Route, leastInterchanges: Route }

### GET /api/v1/metro-updates

**Description:** Fetch current metro updates (delays, closures)

**Response:** { updates: MetroUpdate[] }

### GET /api/v1/stations/:stationId

**Description:** Fetch detailed station information

**Response:** { station: Station with extended details }

### GET /api/v1/crowd-status

**Description:** Fetch real-time crowd levels for stations

**Response:** { crowdData: { stationId: crowdLevel, ... } }

# 7. User Flows

## 7.1 Primary Flow: Route Calculation

11. User launches app
12. Home screen loads with "From" and "To" station inputs
13. User taps "From" field, searchable dropdown appears
14. User types or selects starting station
15. User repeats for "To" station
16. User taps "Search" or "Get Routes" button
17. App shows loading indicator
18. Route results load with two tabs: "Fastest" and "Least Interchanges"
19. User views route summary (time, stations, fare)
20. User scrolls through station list
21. User can tap any station for details (facilities, exit map, crowd level)
22. User can switch tabs to compare "Least Interchanges" route

## 7.2 Alternative Flow: No Route Available

23. User searches for route
24. Metro line is closed due to maintenance or emergency
25. App displays alert: "Route unavailable due to Blue Line closure"
26. App suggests alternative transport: "Try bus route 413 or auto-rickshaw"
27. User can view estimated service resumption time

## 7.3 Settings & Theme Customization

28. User taps Settings (gear icon, top-right)
29. Settings screen loads
30. User sees "Theme" option with toggle
31. User switches from Mocha to Latte (or vice versa)
32. App immediately updates UI colors
33. User closes Settings, theme persists on app restart

# 8. Success Metrics

## 8.1 Key Performance Indicators (KPIs)

### Acquisition

- Downloads in first 3 months: Target 50,000
- App Store rating: Target ≥ 4.5 stars

- Monthly active users (MAU): Target 80% of downloads

### Engagement

- Average sessions per user per day: Target ≥ 1.5
- Average session duration: Target ≥ 3 minutes
- Route searches per session: Track & analyze
- Theme toggle adoption: Target ≥ 20% of users
- Station detail views: Track popularity of features

### Retention

- Day 1 retention: Target ≥ 50%
- Day 7 retention: Target ≥ 35%
- Day 30 retention: Target ≥ 25%
- Churn rate: Target ≤ 5% per month

### Technical

- App crash rate: Target ≤ 0.1%
- Route calculation latency: Target median ≤ 300 ms
- Offline mode success rate: Target 99%
- Real-time update sync success: Target 95%

## 9. Development Roadmap

### Phase 1 (MVP) – Months 1–3

**Goal:** Launch core route planning with offline support

- Implement dual route calculation (Dijkstra's algorithm for fastest, custom algorithm for least interchanges)
- Build home, route results, and station detail screens
- Implement offline caching with SQLite
- Set up real-time update infrastructure (mock data)
- Implement Catppuccin Mocha & Latte themes
- Launch on Google Play Store (closed beta)
- Basic analytics setup

### Phase 2 (Extended Functionality) – Months 4–6

**Goal:** Integrate real-time updates and crowd data

- Connect to real metro delay/closure API
- Integrate crowd level data (from metro authority or inferred model)

- Implement alternative transport suggestions
- Add exit map images for major stations
- Implement facility icons & operating hours
- Public release on Google Play Store
- Marketing push (social media, Reddit, local communities)

### Phase 3 (Enhancements) – Months 7–9

**Goal:** Optimize and expand user experience

- Implement saved routes (requires authentication)
- Add user feedback/ratings system
- Performance optimization (reduce cache size, faster routing)
- Implement accessibility features (step counts, elevator locations)
- Analyze user data & implement optimizations
- Consider multi-language support

### Phase 4+ (Scaling & New Modes) – Beyond

- iOS port
- Integration with Google Maps, Apple Maps
- Integration with auto-rickshaw & bus APIs
- Multi-city support (other metro systems)
- Monetization (ads, premium features, API partnerships)

## 10. Appendix

### 10.1 Delhi Metro Line Colors Reference

(Official Delhi Metro Rail Corporation colors to be used for line indicators in the app)

| | | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

### 10.2 Terms & Definitions

**Interchange:** A point where a commuter must change metro lines

**Fastest Route:** The route with minimum total travel time (including wait times at interchanges)

**Least Interchanges:** The route with fewest line transfers

**Real-Time Update:** Live notification of metro delays, closures, or schedule changes

**Offline Mode:** App functionality without internet; uses cached data

**Catppuccin:** A pastel color palette with Mocha (dark) and Latte (light) variants

**MAU:** Monthly Active Users

## 10.3 Future Considerations

- Machine learning for crowd prediction based on historical data
- Voice-guided navigation (audio cues at interchanges)
- AR visualization of metro network
- Gamification (achievements, badges for milestones)
- Community-driven updates (user-reported delays, hazards)
- Integration with Delhi Metro's official APIs for seamless data sync

## 10.4 Assumptions & Constraints

### Assumptions

- Delhi Metro publishes or provides API access to network data
- Real-time delay/closure data will be available from metro authority
- Crowd data can be inferred from historical or real-time sensor data (or crowdsourced)
- Users have basic Android device literacy

### Constraints

- App limited to Delhi Metro system in Phase 1
- Minimum Android 8.0 API level 26
- No backend server required initially (local caching)
- Initial budget assumes solo or small-team development
- No monetization in Phase 1 (free app)

## 10.5 References

Delhi Metro Rail Corporation (DMRC) Official Website: dmrcdelhi.com

Catppuccin Color Palette: catppuccin.com

Android Jetpack Documentation: developer.android.com/jetpack

Material Design 3: m3.material.io

Graph Routing Algorithms: en.wikipedia.org/wiki/Dijkstra's_algorithm

## 10.6 Document Version History

| | | | |
|---|---|---|---|

| | | | |
|---|---|---|---|
| | | | |

— End of Document —